SYSTEM REQUIRMENT SPECIFICATIONS

Customizable SAAS platform with LLM-Powered Analytics

PURPOSE OF THE SYSTEM

This Software Requirements Specification (SRS) establishes the functional and non-functional specifications for a SaaS solution designed for micro, home-based, and small B2C sellers. The solution empowers users to create personalized storefronts, efficiently manage inventory and orders, and leverage AI-driven business intelligence. All critical data is maintained in a PostgreSQL database, with actionable recommendations generated by a Large Language Model (LLM), accessed securely via the Model Context Protocol (MCP).

PROBLEMS IN THE EXISTING SYSTEM:

Most SaaS e-commerce platforms offer only basic, template-driven customization for seller homepages and product displays. Deep brand personalization or unique workflows are often restricted, requiring costly upgrades or development

Sellers have limited ownership of the data they generate. With SaaS, the platform provider maintains control; if policies change or accounts get suspended, access to essential business data can be compromised

With restricted customization comes bland storefronts—many sellers end up with nearly identical shopfronts, making it difficult to stand out or implement unique customer experiences

Default dashboards mostly offer basic, historical reporting—lacking real-time insights, anomaly detection, or cross-metric correlation. This leaves sellers blind to subtle issues or unable to proactively spot and act on opportunities

Integrating with legacy systems, external tools, or AI/LLM components is often challenging. Such integrations can be costly, inflexible, and sometimes impossible to implement on top of SaaS-locked platforms

SOLUTION OF THESE PROBLEMS

Provides fully customizable, seller-specific homepages with drag-and-drop or flexible layout options allowing unique brand presentation rather than fixed templates.

Stores all key business data securely in your PostgreSQL/MySQL database owned and accessible by the seller; integration via MCP ensures controlled, auditable AI access.

Integrates a Large Language Model via MCP to analyze live sales and inventory data, providing real-time, actionable insights, trend detection and anomaly alerts.

Uses Model Context Protocol (MCP) for a standardized, secure AI-to-database interface allowing seamless integration of LLM-driven intelligence with minimal custom plumbing.

Employs secure LLM access only through MCP, ensuring privacy and compliance; provides sellers simple natural-language insights without complex coding or AI management needs.

SCOPE OF THE PROJECT

This project delivers a Software as a Service (SaaS) solution enabling micro, home-based, and small B2C sellers to create, customize and manage personalized online storefronts. Sellers can easily design their homepage, arrange products, banners, promotions without technical expertise. All critical business data—products, orders, inventory and customer details—is securely maintained in a PostgreSQL or MySQL relational database. This ensures strong data integrity, consistency and seller ownership of their business information. The platform integrates a Large Language Model (LLM) through the Model Context Protocol (MCP), providing sellers with real-time, context-aware analytics, actionable recommendations, and trend detection based on live transactional data. Sellers manage product catalogs, inventory and orders through an intuitive dashboard, benefiting from automated insights and alerts driving better business decisions.

FUNCTIONAL COMPONENTS OF THE PROJECT:

There are registered users in the system: **Sellers**, **Customers** . A Seller manages their own storefront, products, and orders. A Customer is an end-user who purchases from a Seller and Integration with a Large Language Model (LLM) via the Model Context Protocol (MCP)

1. **Seller Dashboard and Storefront Customization**
   This component enables sellers to onboard themselves, create and personalize their online storefronts—arranging products, banners, and promotional content easily through an intuitive interface without coding.

2. **A Seller should be able to:**

- Login to the system through the main seller portal.
- Change their password and update their profile information after logging in.
- See their store's key performance metrics on a dashboard (like total sales, number of orders, and top-selling products).
- Query the current inventory level for any of their products.
- See their complete order and sales history since they joined the platform.
- Manage their product catalog by adding, editing, or removing products, specifying details like name, price, description, images, and inventory count.
- Customize their storefront's homepage by arranging products, adding promotional banners, and editing content.
- See their current active orders and any pending notifications submitted by the system (e.g., low-stock alerts, new AI-driven insights).
- Process customer orders by updating their status (e.g., Confirmed, Shipped, Delivered, Cancelled).
- Withdraw or deactivate a product listing temporarily from their storefront.
- Create or cancel a promotional offer (e.g., a discount code). This action would reflect instantly on their storefront.
- Get help on how to use the different features of the platform through an integrated support AI agent

3. Automated AI-Driven Insights:
Actionable business insights, powered by the LLM, will be automatically generated from the seller's sales and inventory data in the PostgreSQL /MySQL database. A notification regarding new key insights (e.g., "Product X is trending," "Consider bundling Product X ") will be sent to sellers automatically.

4. Periodic Summary Reports:
A summary report of sales performance, top-selling products, inventory status, and recent customer activity will be sent to every seller periodically

<u>STUDY OF THE SYSTEM</u> :

The current e-commerce landscape for micro, home-based, and small B2C sellers is dominated by SaaS platforms that, while easy to set up, present significant limitations. Existing systems often force sellers into rigid, template-based storefronts, which restricts brand differentiation and offers minimal customization. Furthermore, sellers typically lack true ownership and control over their business data, face escalating hidden costs with growth, and are provided with basic, retrospective analytics that offer little predictive or actionable value.

Integrating advanced technologies like AI for business intelligence is often complex, expensive, and outside the technical reach of these non-technical users. The primary problem is a market gap: a lack of an affordable, scalable, and intelligent e-commerce solution that provides deep customization, data ownership and powerful, real-time analytics to empower small sellers.

User Characteristics:

Sellers: The main users of the platform. They are micro, small, or home-based business owners with minimal technical expertise. They will use the system to manage their entire online business—from storefront design to order fulfillment and performance analysis.

Customers: The end-users who visit the sellers' storefronts to browse and purchase products. The system provides them with a simple, secure, and seamless shopping experience.

Proposed System and Objectives

To address these challenges, this project proposes the development of a multi-tenant SaaS e-commerce solution specifically designed for micro and home-based sellers. The system will provide sellers with the tools to build and manage a fully personalized online storefront, moving beyond the limitations of generic templates.

The core architecture is built on a robust **relational databases**, which serves as a centralized and secure repository for all seller data, including products, inventory, orders, and customer information. This ensures data integrity and provides sellers with ownership of their business intelligence.

The key innovation of the proposed system is the integration of a **Large Language Model (LLM)** via the **Model Context Protocol (MCP)**. This allows the platform to securely connect to the live PostgreSQL/MySQL data and deliver real-time, AI-driven insights. Sellers receive actionable recommendations—such as product bundling suggestions, sales trend alerts, and inventory forecasts—directly on their dashboard in plain, easy-to-understand language.

The primary objectives of the system are:

Provide an intuitive, no-code interface for creating a unique and highly personalized brand presence.

Leverage AI to transform raw sales data into strategic insights that drive business growth.

Give sellers control over their business data with a secure, reliable backend.

Streamline product, inventory, and order management through a single, user-friendly dashboard.

INPUT / OUTPUT

Inputs:

- **Seller** enters their user ID and password to log in.
- **Seller** customizes their storefront layout, uploads banners, and arranges products.
- **Seller** adds or edits product details, including name, price, description, images, and inventory count.
- **Seller** updates the status of a customer's order (e.g., Confirmed, Shipped, Delivered).
- **Seller** asks a natural language question to the AI for business insights (e.g., "What were my top-selling items last week?").
- **Customer** browses the storefront and adds items to their shopping cart.
- **Customer** provides their name, contact info, and delivery address during checkout.
- **Customer** submits a review and a rating for a product they purchased.

Outputs:

- The system displays the **seller's** personalized dashboard upon successful login.
- The **seller's** public-facing storefront is updated instantly to reflect layout changes and new products.
- New product information, order details, and customer data are stored securely in the PostgreSQL database.
- The system displays the current status of all orders on the seller's dashboard.
- The system displays real-time, AI-generated answers, reports, and actionable suggestions to the **seller**.
- The system displays an order confirmation page to the **customer** after a successful purchase.
- An automated email notification is sent to the **customer** for order confirmation and shipping updates.
- The system displays product reviews and ratings on the relevant product pages.

PERFORMANCE REQUIREMENTS:

Performance is a critical measure of the output and user experience provided by this SaaS application. A clear specification of these requirements is essential during the analysis phase to ensure the final system is robust, responsive, and meets the operational demands of both sellers and their customers. Since the platform is designed to replace inefficient or manual existing systems, its performance must be demonstrably superior to justify its adoption. It is very difficult to re-engineer performance characteristics once the system has been designed, and a platform that does not meet the performance expectations of its users will fail to retain them.

The performance requirements for this system can be stated as follows:

- **Response Time and Latency:** The system must be highly responsive to user interactions. Key performance indicators include:
    - Seller dashboard and customer-facing storefront pages should load in under 3 seconds on a standard internet connection.
    - Database queries for product information or order history must return results in under 2 seconds.
    - Real-time actions, such as adding a product to the cart or updating inventory, should be reflected in the system almost instantaneously (under 500ms).
- **Scalability and Concurrency:** The platform must be able to handle a growing number of sellers and their customers without degradation in performance. It should support hundreds of concurrent user sessions per seller during peak traffic events (e.g., a promotional sale) and maintain performance as the total number of sellers on the platform grows into the thousands.
- **Data Accuracy and Integrity:** The system must ensure complete accuracy in all its operations. Inventory counts must be updated in real-time with every sale to prevent overselling. Financial calculations for order totals, discounts, and seller payouts must be precise. All data stored in the PostgreSQL database must maintain its integrity.
- **AI Insight Performance:** The AI-driven agents and the LLM analytics must provide timely and relevant insights.
    - Requests for simple, natural language business queries should be answered within 5-10 seconds.
    - Automated insights and alerts (e.g., low-stock warnings) must be generated and delivered to the seller with minimal delay from the triggering event.

FEASIBILITY REPORT

A study was conducted to assess the feasibility of the proposed system across three key areas:

- **Technical Feasibility:** The project is technically feasible. It relies on a stack of mature, widely-adopted, and well-documented technologies, including PostgreSQL, Node.js, React/Next.js, and established cloud hosting platforms. The integration of an LLM via the Model Context Protocol (MCP) represents a modern but well-defined architectural pattern for securely connecting AI to live data sources. The modular design of the system also ensures that components can be developed, tested, and deployed independently, reducing complexity and risk.
- **Operational Feasibility:** The system is designed with a high degree of operational feasibility. Its target users—non-technical sellers—are the central focus of the user experience. The platform will feature an intuitive dashboard, guided onboarding, and automated insights, which minimizes the need for specialized training or

technical skills. By automating complex data analysis and presenting it in a simple format, the system ensures that sellers can easily incorporate it into their daily operations.

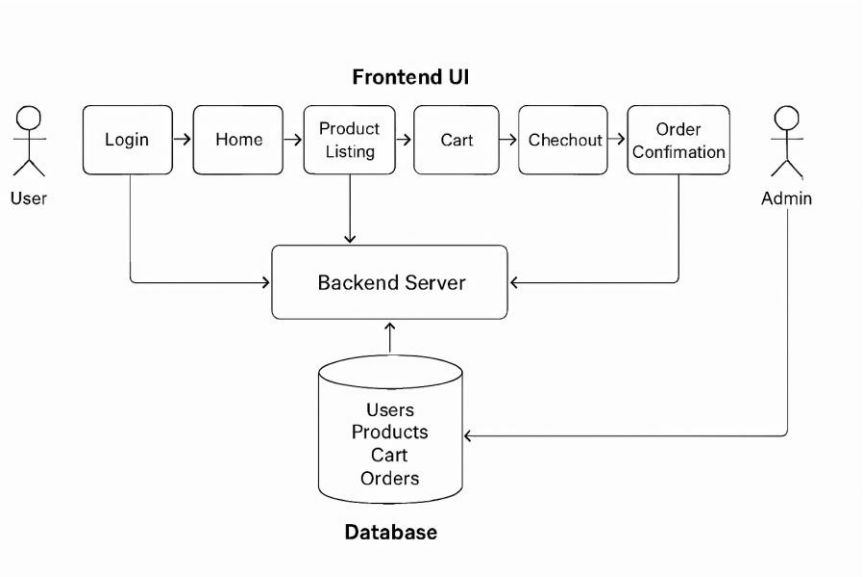
- **Economic Feasibility:** The project is economically feasible. By utilizing open-source technologies and a SaaS subscription model, the initial development costs are manageable and the barrier to entry for sellers is low. The platform provides a high-value proposition by offering enterprise-grade analytics and customization at a fraction of the cost of developing a bespoke solution. This creates a strong potential for high user adoption and a sustainable revenue model based on recurring monthly or annual subscriptions.

ER Diagram

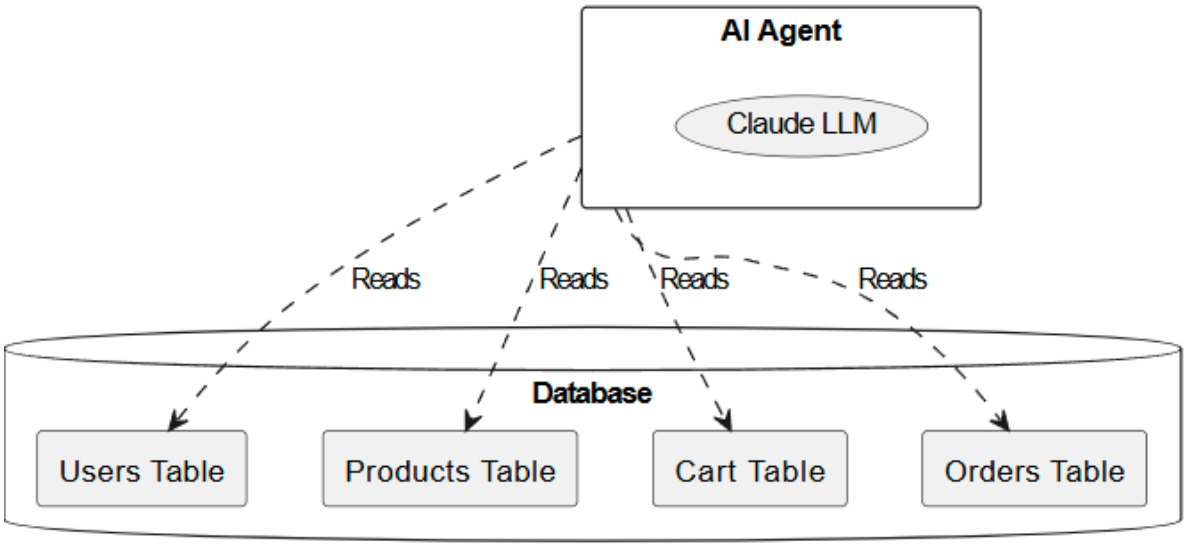
The relationships within the system are structured through a conceptual **ER Diagram**. This diagram not only specifies the core existential entities—such as **Sellers**, **Products**, and **Orders**—but also defines the standard relations and cardinalities that are necessary for the system state to be maintained consistently within the PostgreSQL/MySQL database.

The Entity-Relationship Diagram (ERD) is the notation used to conduct the data modeling activity for this project. The attributes of each data object noted in the ERD provide the blueprint for the database schema.

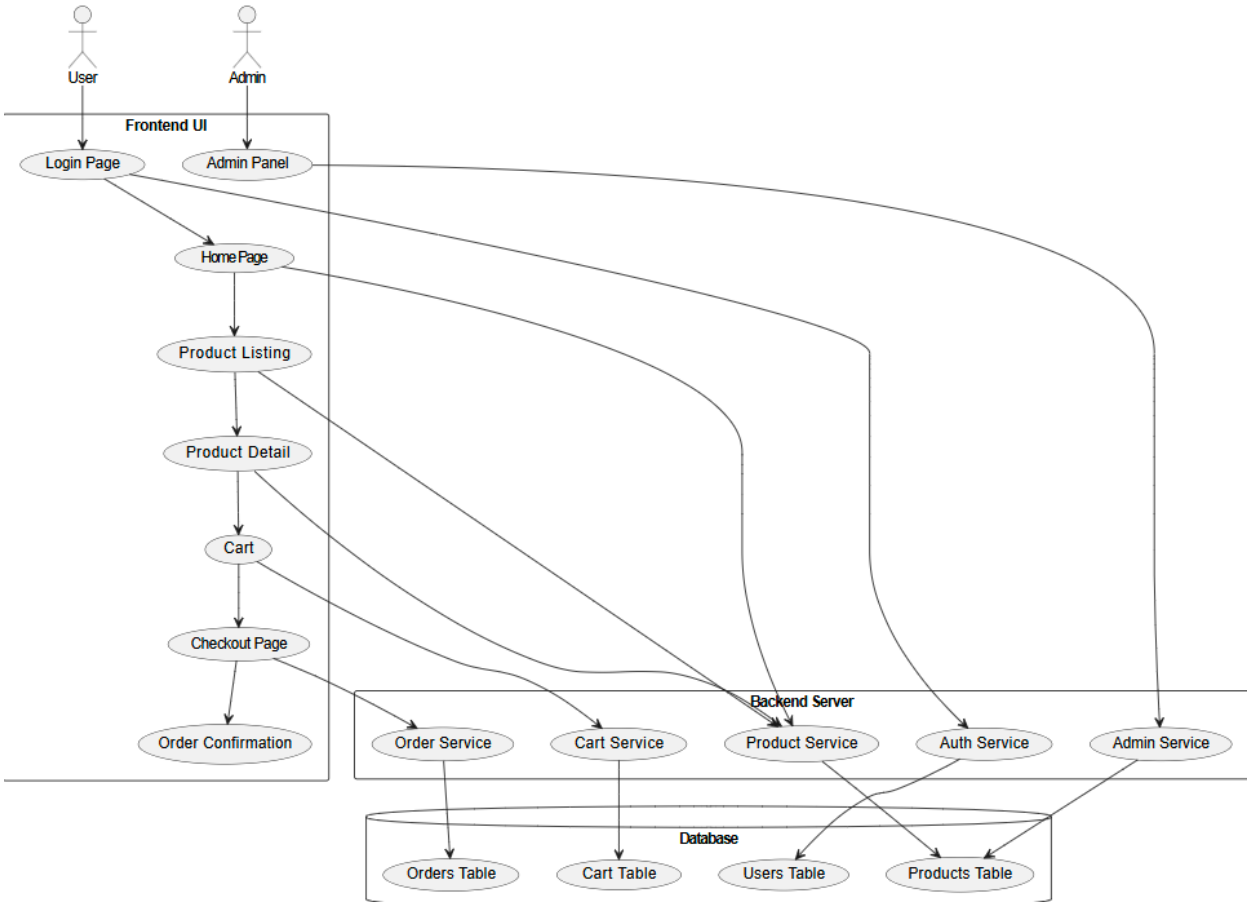
1st dataflow diagrams

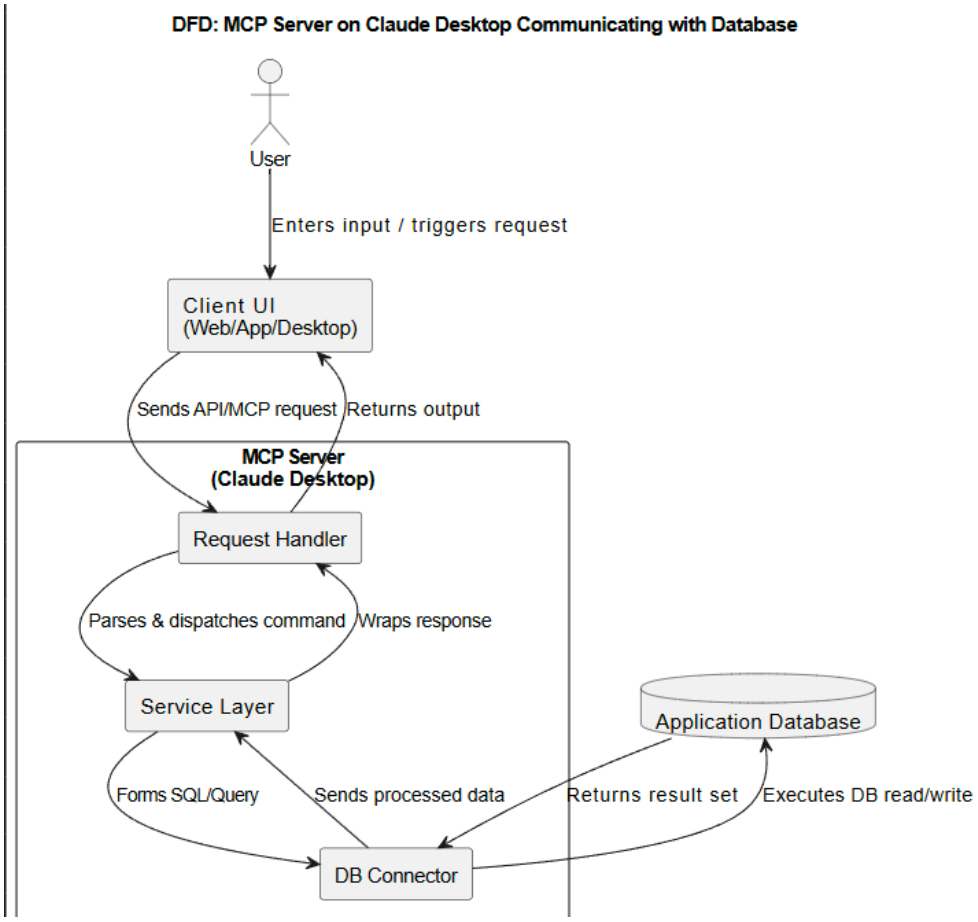


Claude LLM and Database Interaction

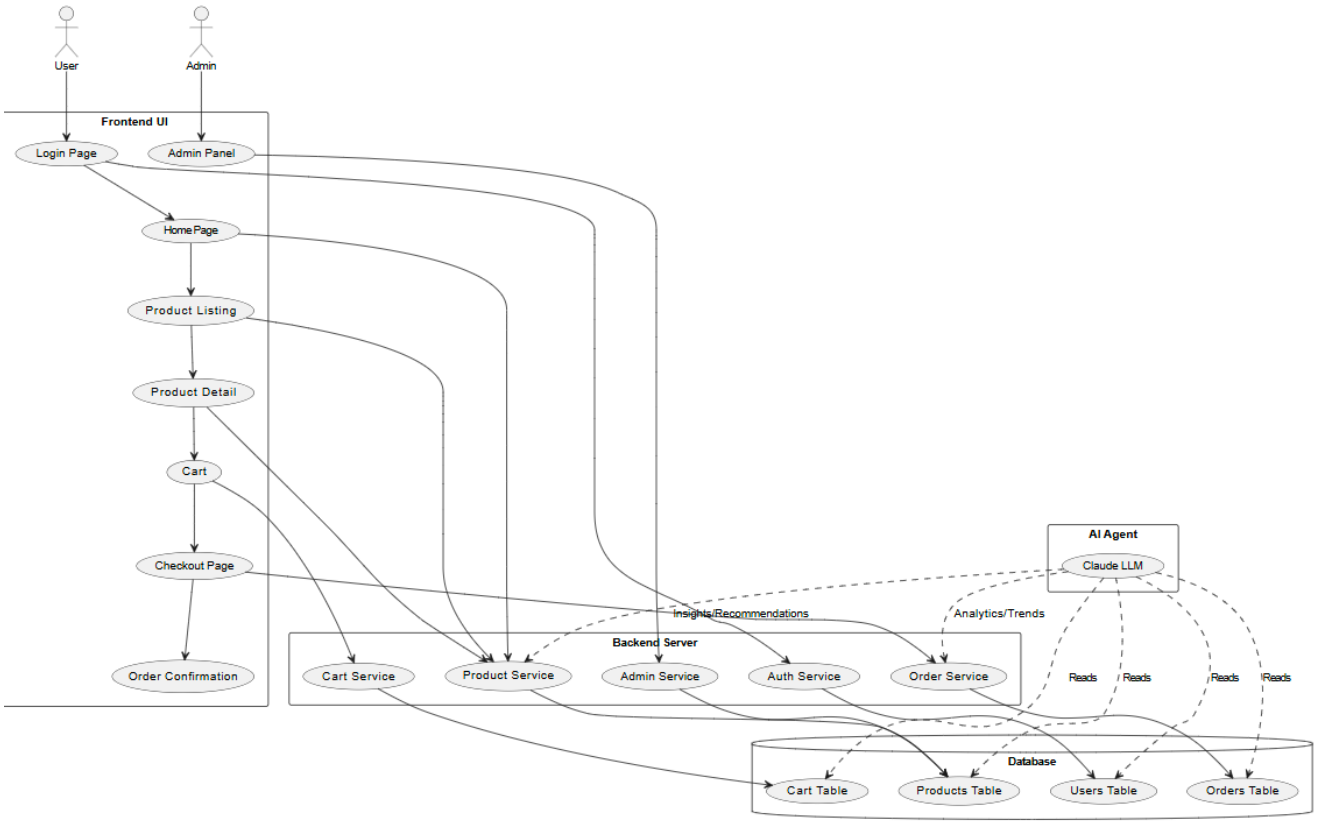


2nd dataflow diagram

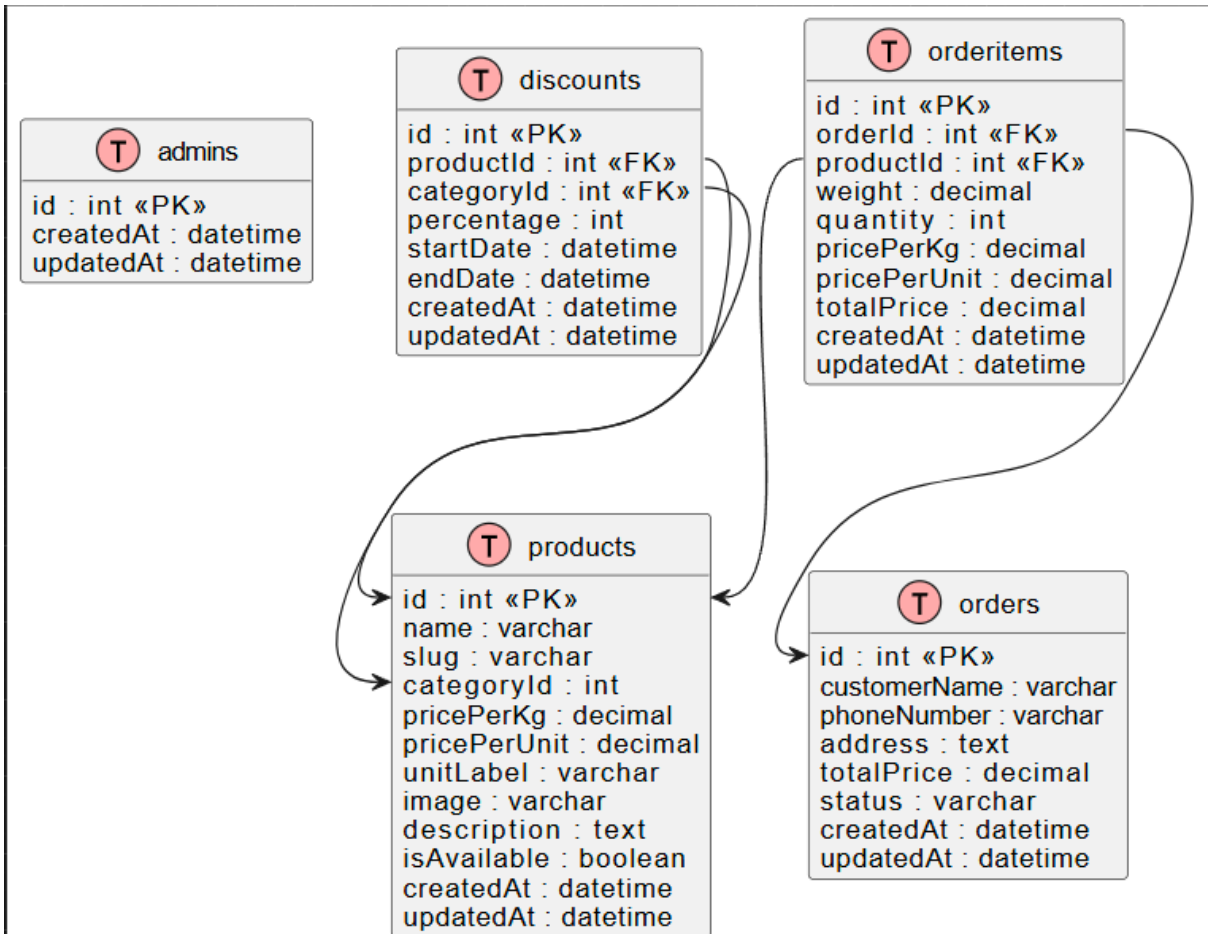




3rd level Data Flow diagram


User
Admin
Frontend UI
Login Page
Admin Panel
Home Page
Product Listing
Product Detail
Cart
Checkout Page
Order Confirmation
AI Agent
Claude LLM
Insights/Recommendations
Analytics/Trends
Backend Server
Cart Service
Product Service
Admin Service
Auth Service
Order Service
Reads
Reads
Reads
Reads
Database
Cart Table
Products Table
Users Table
Orders Table

Database Schema Diagram

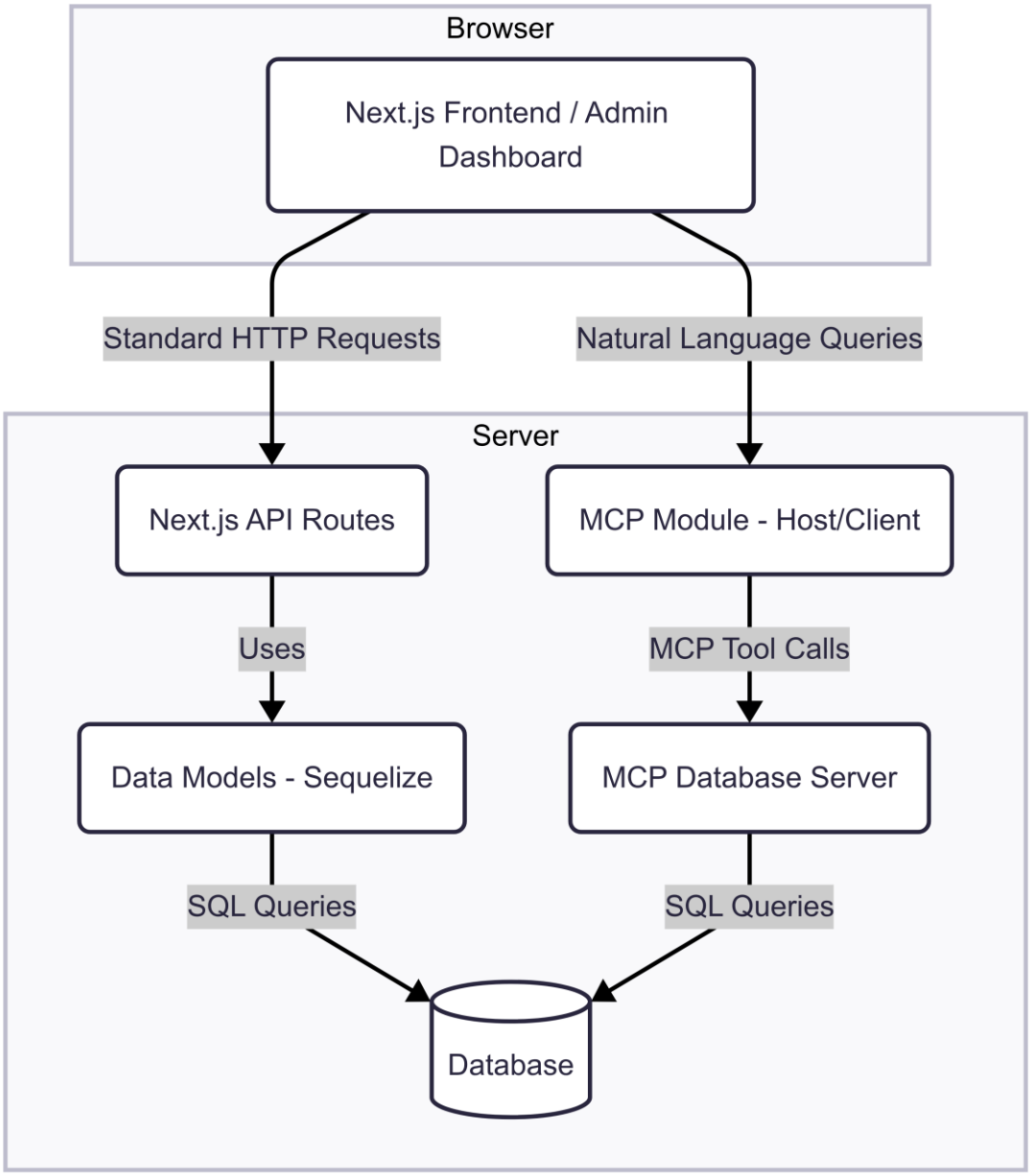


UNIFIED MODELING LANGUAGE DIAGRAMS

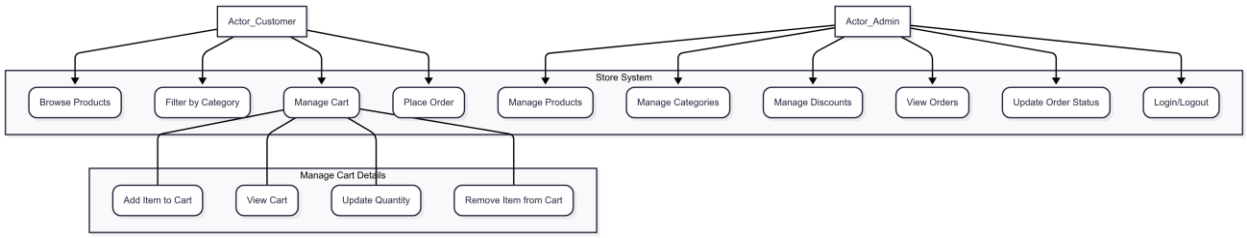
Table of Contents:

Component Diagram:.....2
Sequence Diagrams:.....3
Activity Diagram:.....5
Class Diagram: 6
Collaborative Diagram:.....7

Component Diagram:

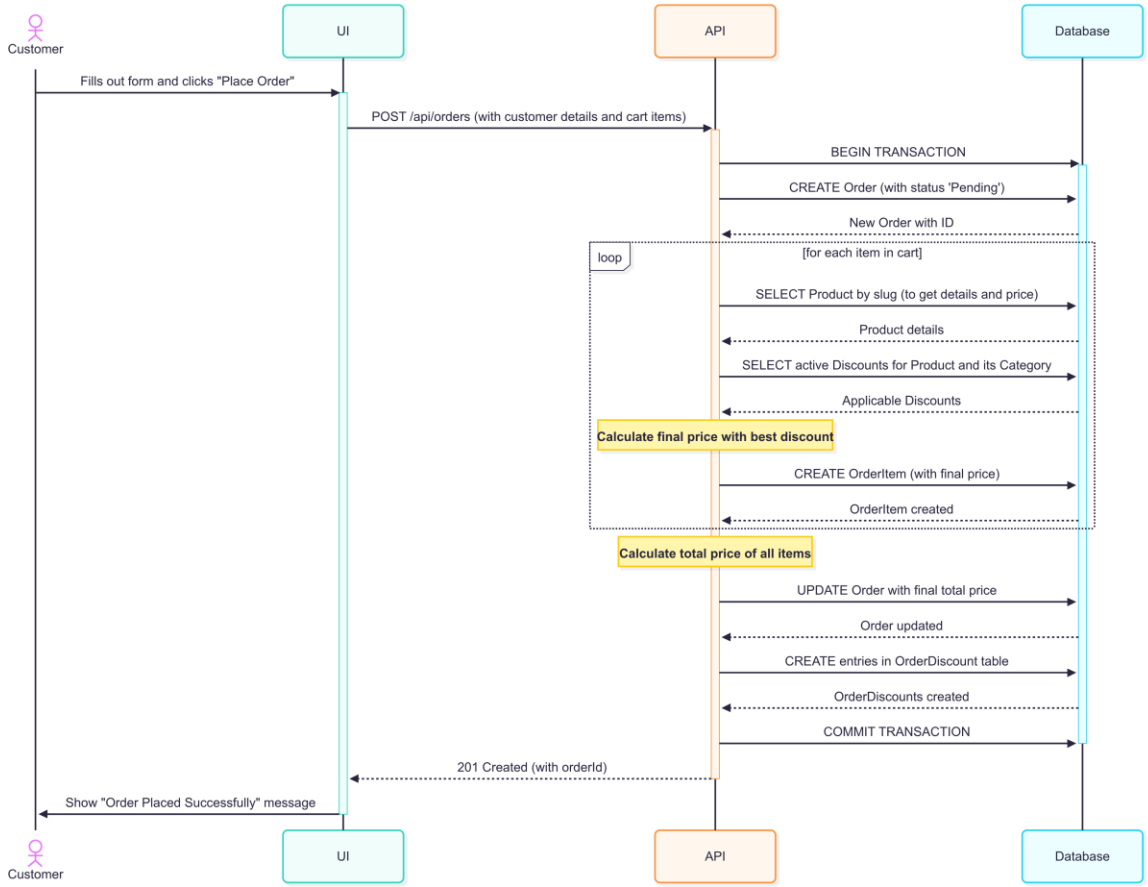


Use Case Diagram:

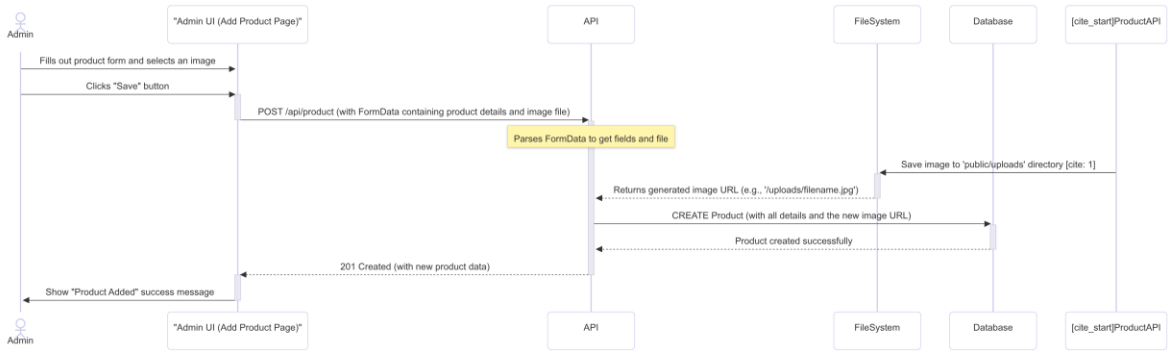


Sequence Diagrams:

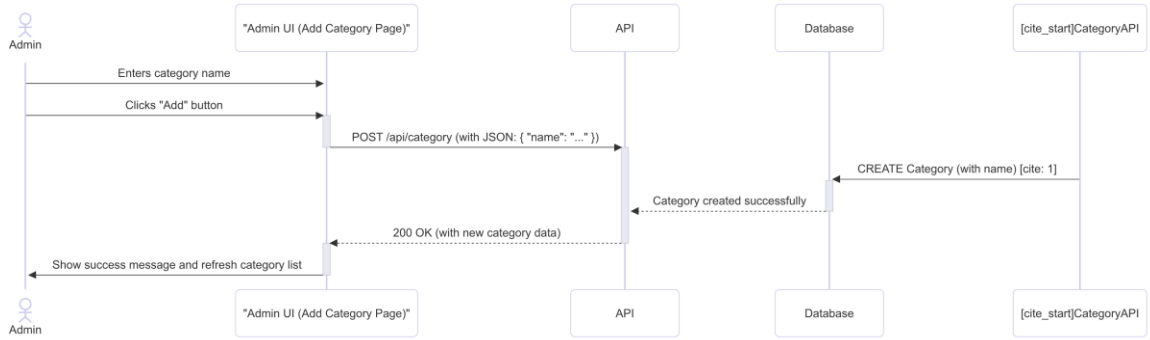
i. Sequence diagram of placing an order



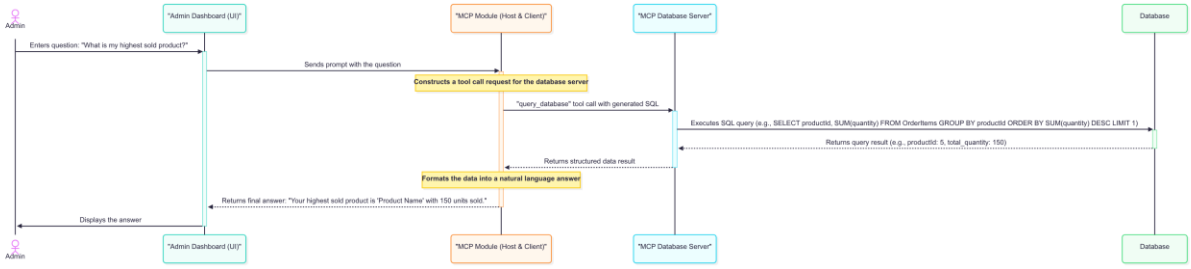
ii. Sequence diagram of adding a product



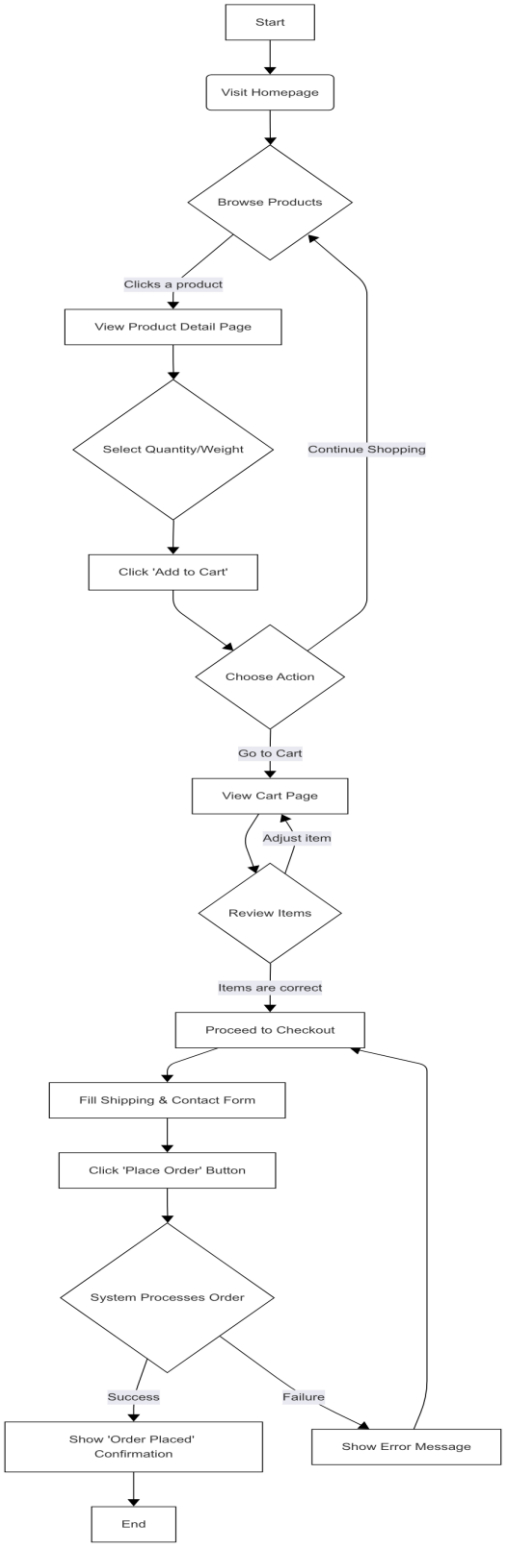
iii. Sequence diagram of adding a category



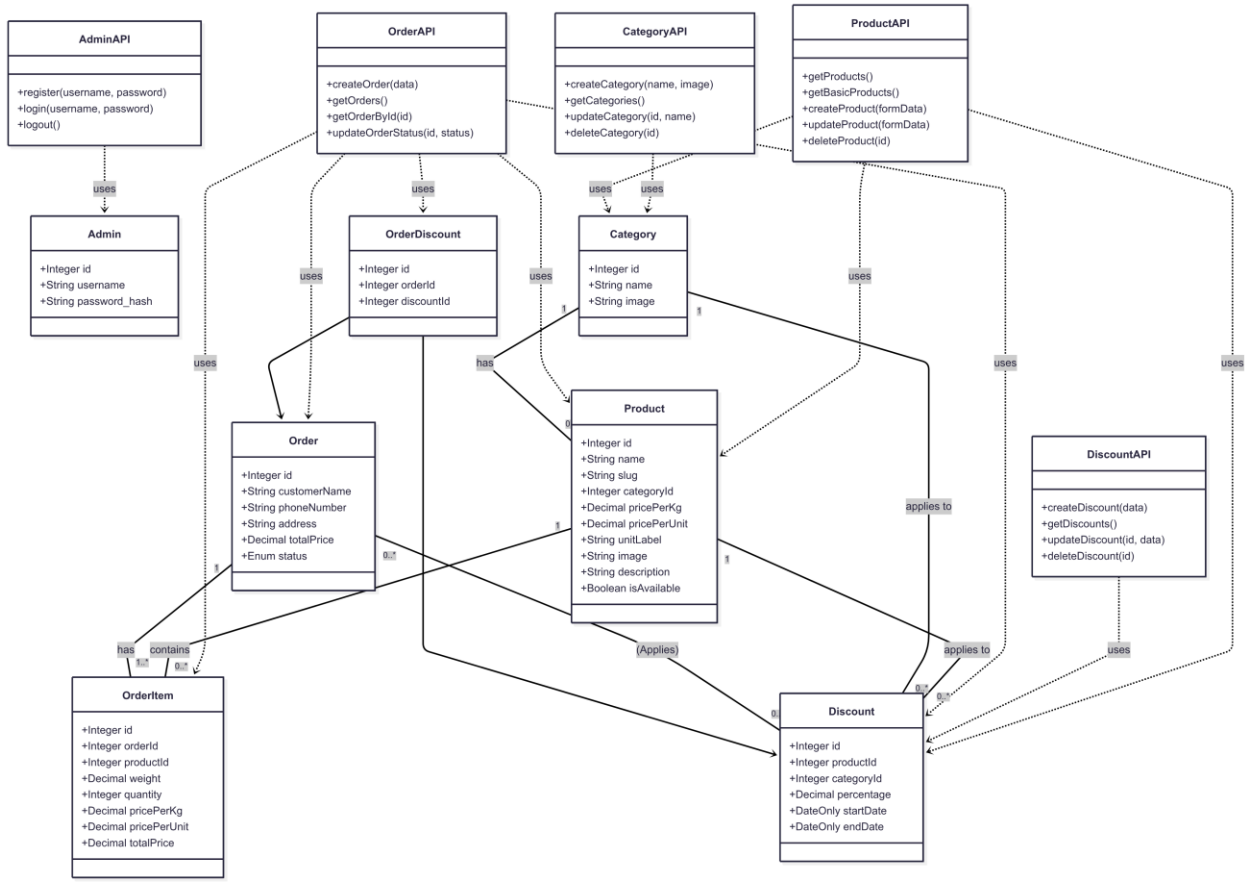
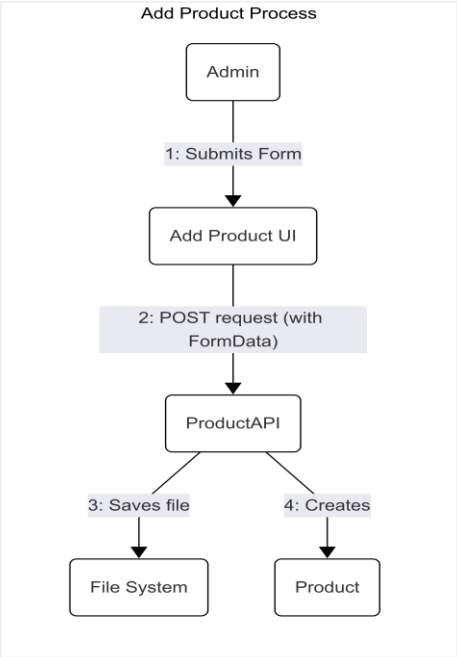
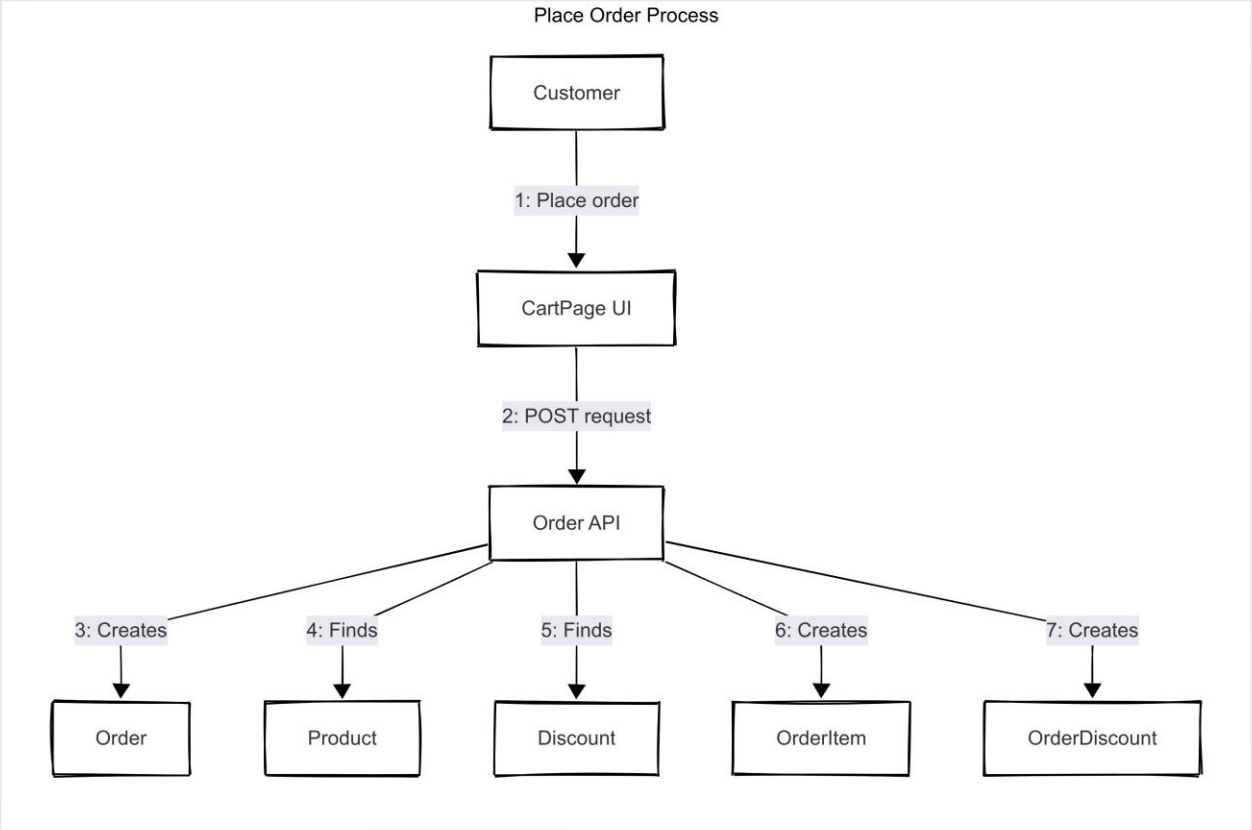
iv. Sequence diagram of user using MCP



Activity Diagram:



Class Diagram:



Collaborative Diagram: